```
>> def describe_value(value)
>>   case value
>>   when be_within(0.1).of(Math::PI) then 'Pi'
>>   when be_within(0.1).of(2 * Math::PI) then 'Double Pi'
>>   end
>> end
=> :describe_value
>> describe_value(3.14159)
=> "Pi"
>> describe_value(6.28319)
=> "Double Pi"
```

RSpec expectations perform the same check internally that Ruby's case statement does: they call === on the object you pass in. That object can be anything—including another matcher.

Passing One Matcher Into Another

It may not be obvious why you'd need to pass a matcher to another matcher. Let's say you expect a particular array to start with a value that's near π. With RSpec, you can pass the be_within(0.1).of(Math::PI) matcher into the start_with matcher to specify this behavior:

```
>> numbers = [3.14159, 1.734, 4.273]
=> [3.14159, 1.734, 4.273]
>> expect(numbers).to start_with( be_within(0.1).of(Math::PI) )
=> true
```

It just works! Now, let's see what the message looks like when the expectation fails:

```
>> expect([]).to start_with( be_within(0.1).of(Math::PI) )
RSpec::Expectations::ExpectationNotMetError: expected [] to start with be ↩
within 0.1 of 3.141592653589793
    « backtrace truncated »
```

Unfortunately, the failure message is the grammatically awkward "expected [] to start with be within 0.1 of π." Luckily, RSpec provides aliases (generally in the form of a *noun phrase*) for the built-in matchers that read much better in situations like these:

```
>> expect([]).to start_with( a_value_within(0.1).of(Math::PI) )
RSpec::Expectations::ExpectationNotMetError: expected [] to start with a ↩
value within 0.1 of 3.141592653589793
    « backtrace truncated »
```

Much better. This actually reads like English, and is much more intelligible: "expected [] to start with a value within 0.1 of π."